

Threads, SMP, and Microkernels

Chapter 4



Process

- Resource ownership - process includes a virtual address space to hold the process image
- Scheduling/execution- follows an execution path that may be interleaved with other processes
- These two characteristics are treated independently by the operating system



Process

- Dispatching is referred to as a thread or lightweight process
- Resource of ownership is referred to as a process or task



Multithreading

- Operating system supports multiple threads of execution within a single process
- MS-DOS supports a single thread
- UNIX supports multiple user processes but only supports one thread per process
- Windows, Solaris, Linux, Mach, and OS/2 support multiple threads

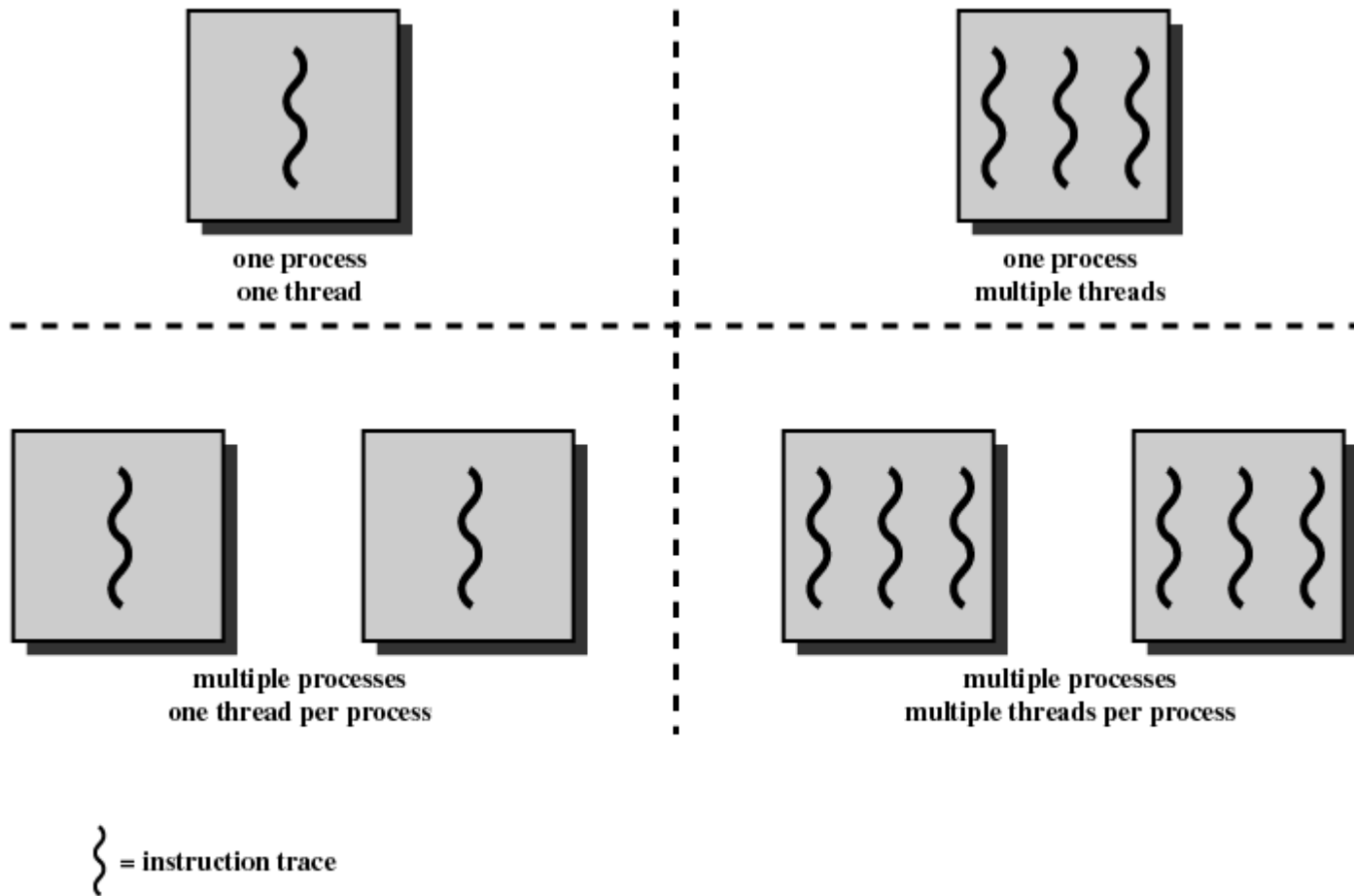
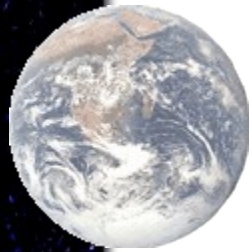


Figure 4.1 Threads and Processes [ANDE97]



Process

- Have a virtual address space which holds the process image
- Protected access to processors, other processes, files, and I/O resources



Thread

- An execution state (running, ready, etc.)
- Saved thread context when not running
- Has an execution stack
- Some per-thread static storage for local variables
- Access to the memory and resources of its process
 - all threads of a process share this

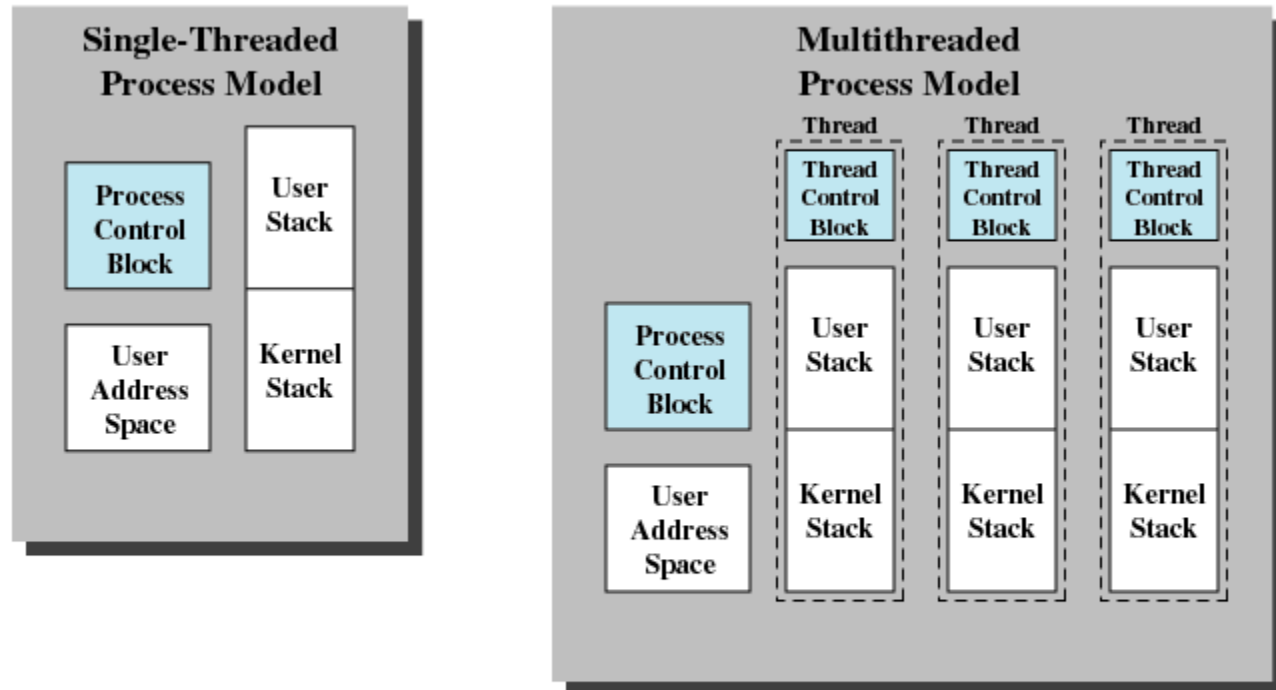
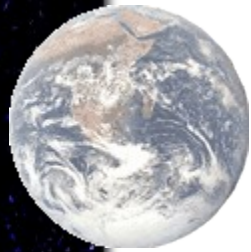


Figure 4.2 Single Threaded and Multithreaded Process Models



Benefits of Threads

- Takes less time to create a new thread than a process
- Less time to terminate a thread than a process
- Less time to switch between two threads within the same process
- Since threads within the same process share memory and files, they can communicate with each other without invoking the kernel



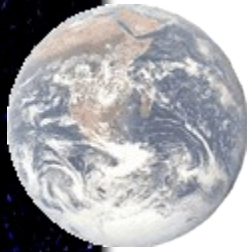
Uses of Threads in a Single-User Multiprocessing System

- Foreground to background work
- Asynchronous processing
- Speed of execution
- Modular program structure



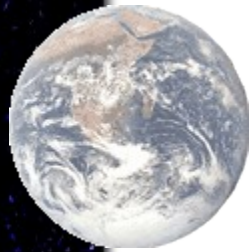
Threads

- Suspending a process involves suspending all threads of the process since all threads share the same address space
- Termination of a process, terminates all threads within the process

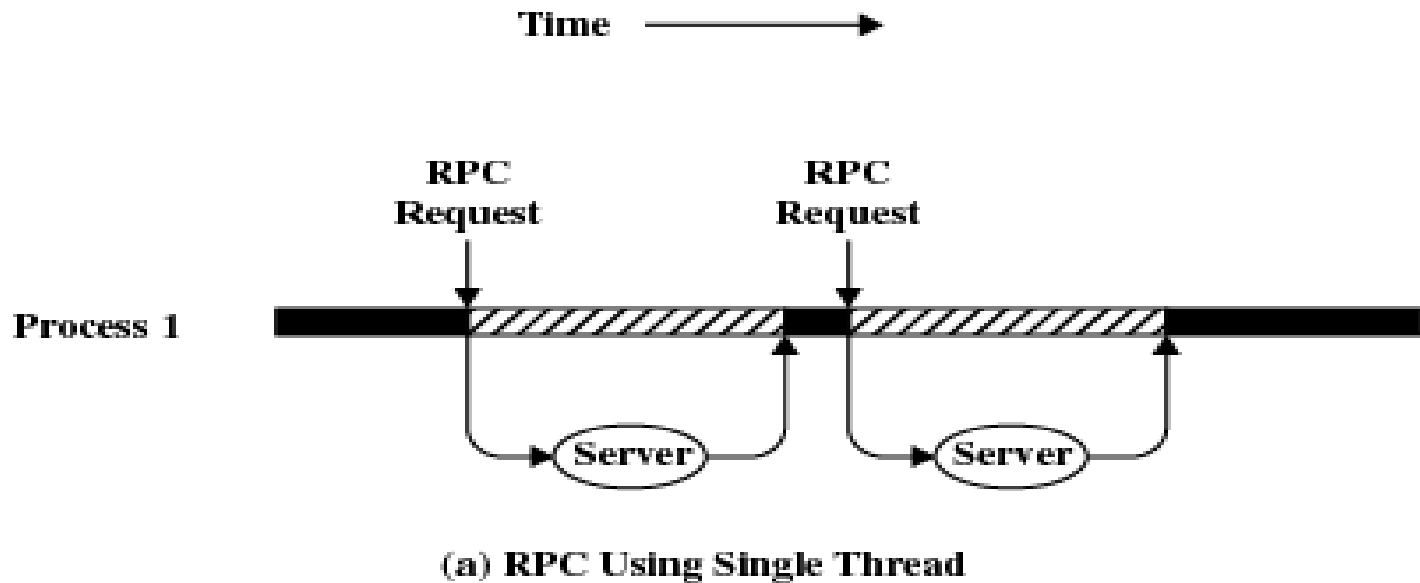


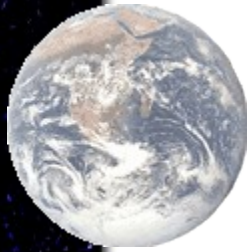
Thread States

- States associated with a change in thread state
 - Spawn
 - Spawn another thread
 - Block
 - Unblock
 - Finish
 - Deallocate register context and stacks



Remote Procedure Call Using Single Thread





Remote Procedure Call Using Threads

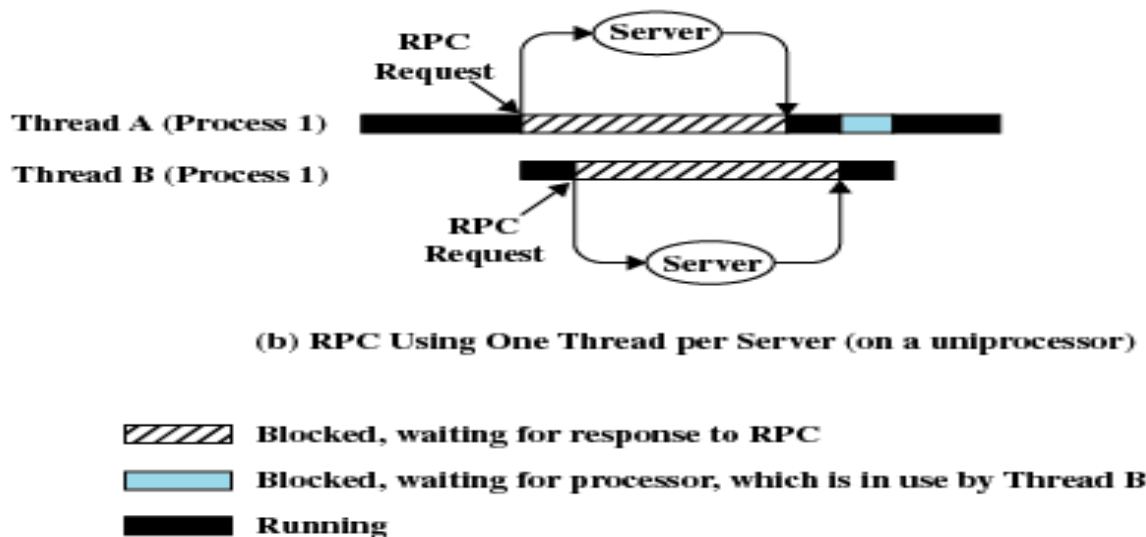
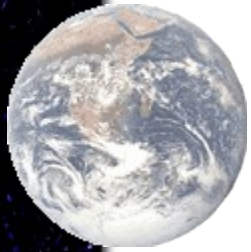


Figure 4.3 Remote Procedure Call (RPC) Using Threads



Multithreading

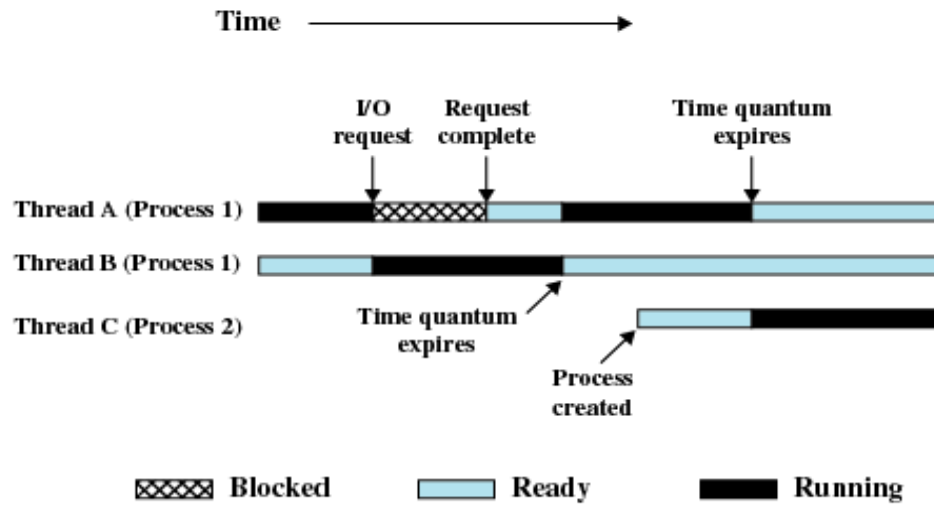
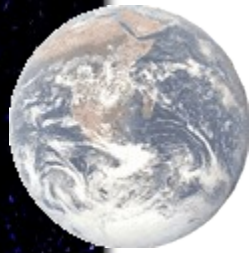


Figure 4.4 Multithreading Example on a Uniprocessor



Adobe PageMaker

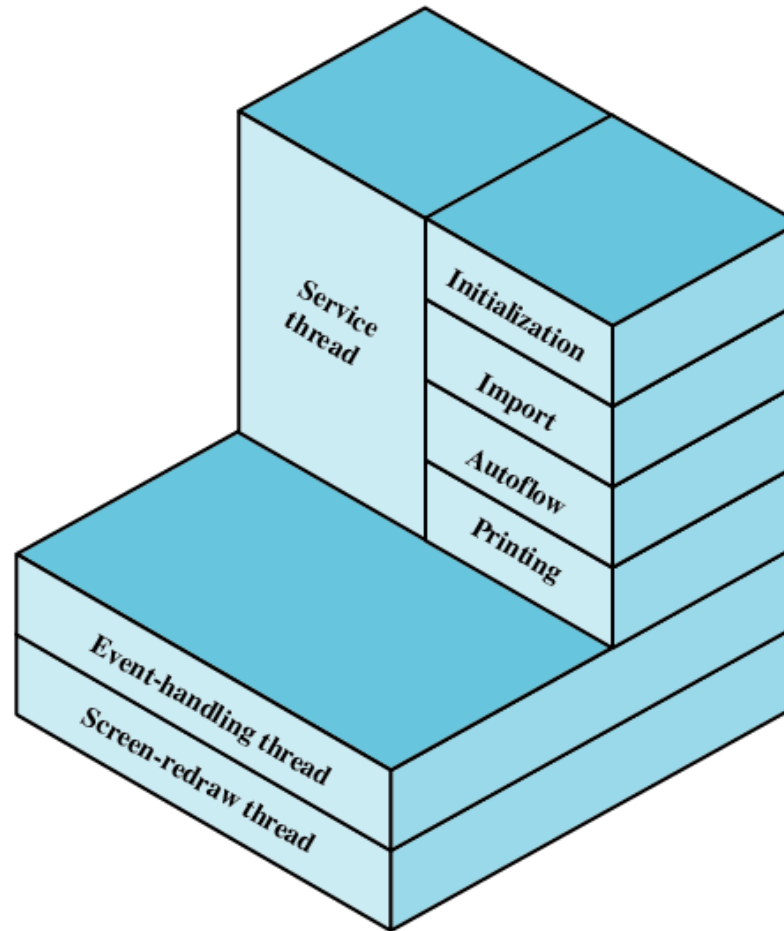
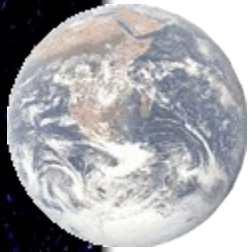
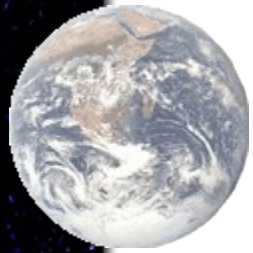


Figure 4.5 Thread Structure for Adobe PageMaker

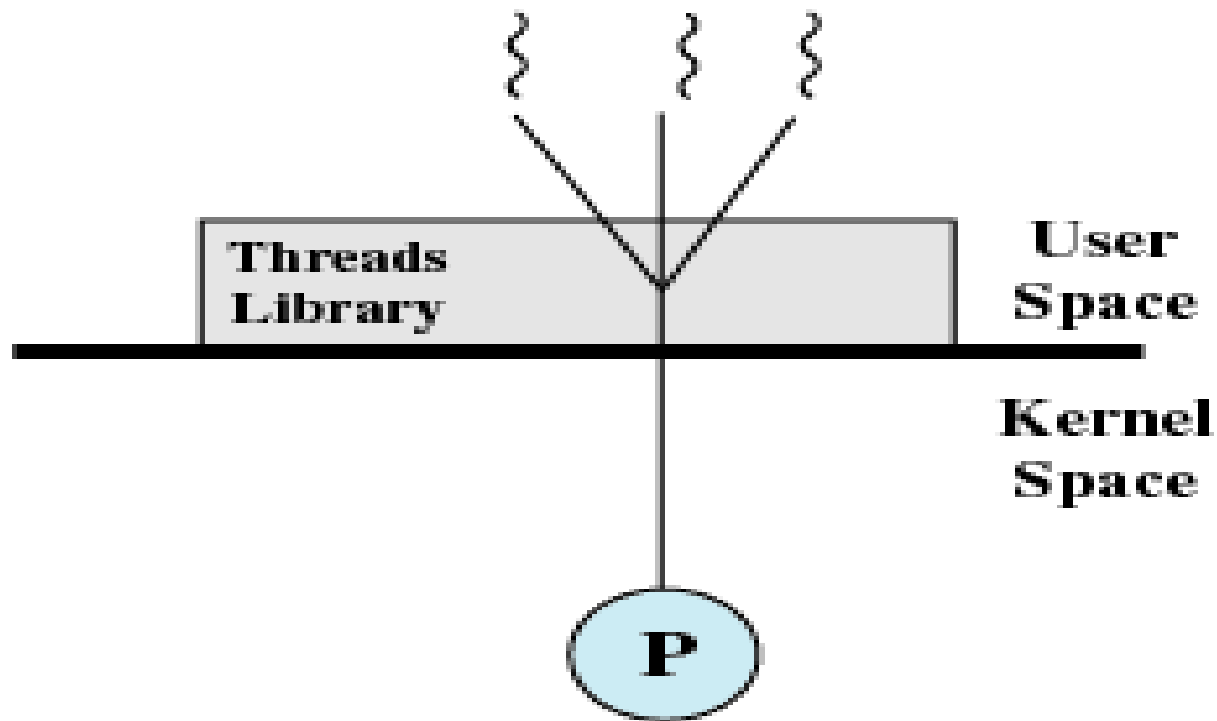


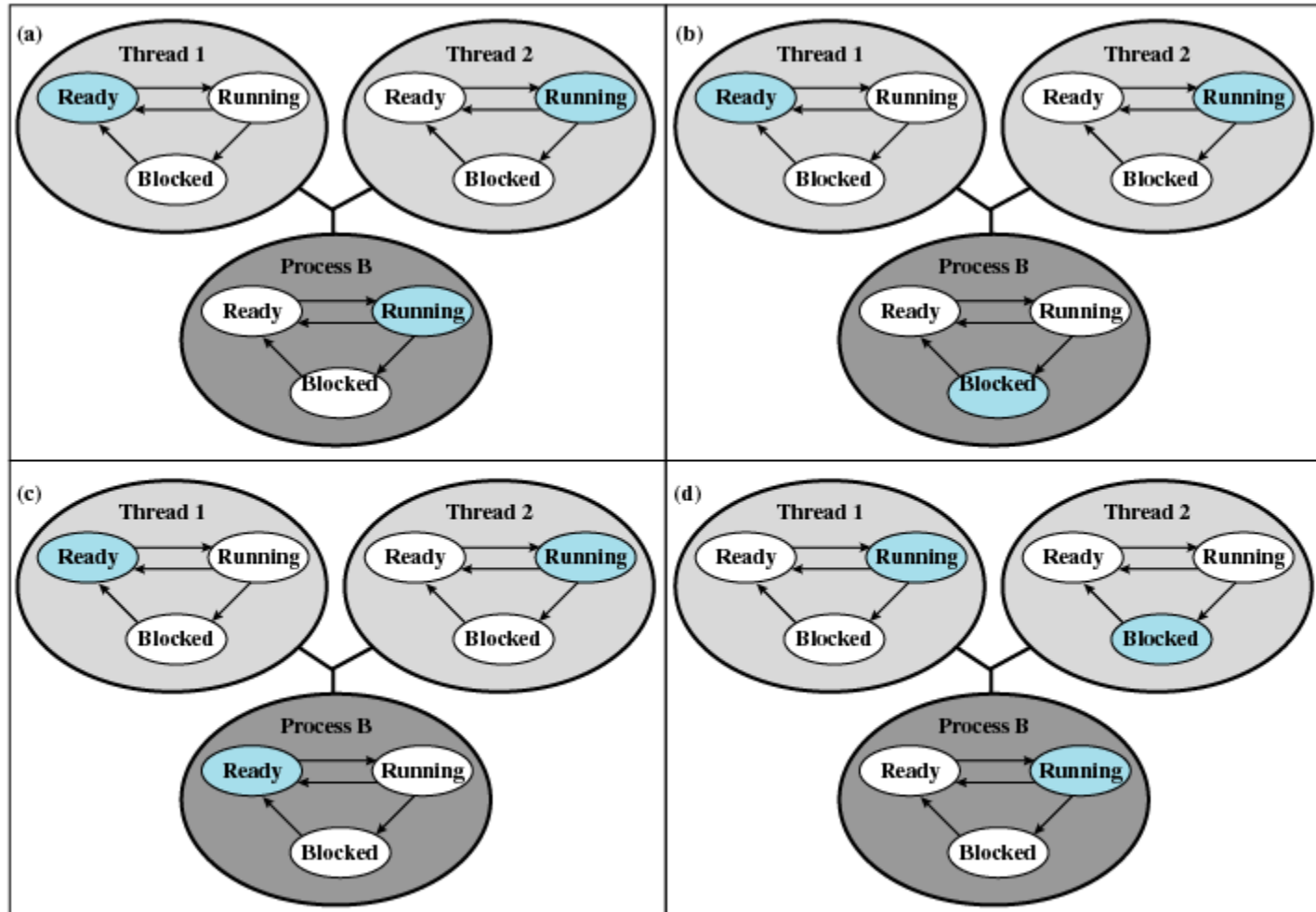
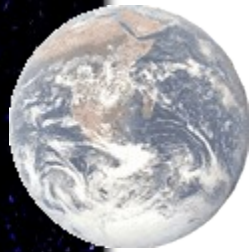
User-Level Threads

- All thread management is done by the application
- The kernel is not aware of the existence of threads



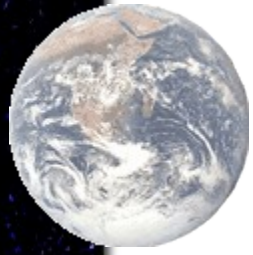
User-Level Threads





Colored state
is current state

Figure 4.7 Examples of the Relationships Between User-Level Thread States and Process States



Advantages of ULT's

- Saves overhead of two mode switches
- **Thread switching do not need mode switching**
- Scheduling can be application specific
- Can run on any O.S.



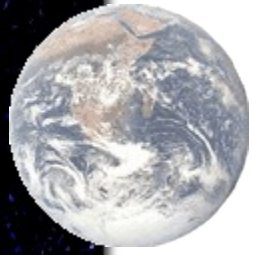
Disadvantages of ULT's

- ULT's executes a system call (Blocking), not only that thread but all the threads within the process are blocked
- Multithreaded application cannot take advantage of multiprocessing.



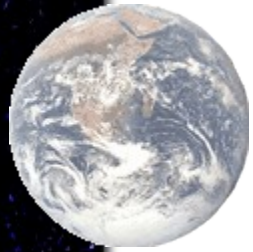
Remedy to disadvantages of ULT's

- Another way to overcome the problem of blocking threads is to use a technique referred to as **jacketing** .
- The purpose of jacketing is to convert a blocking system call into a nonblocking system call.
- instead of directly calling a system I/O routine,
- A thread calls an application-level I/O **jacket** routine.

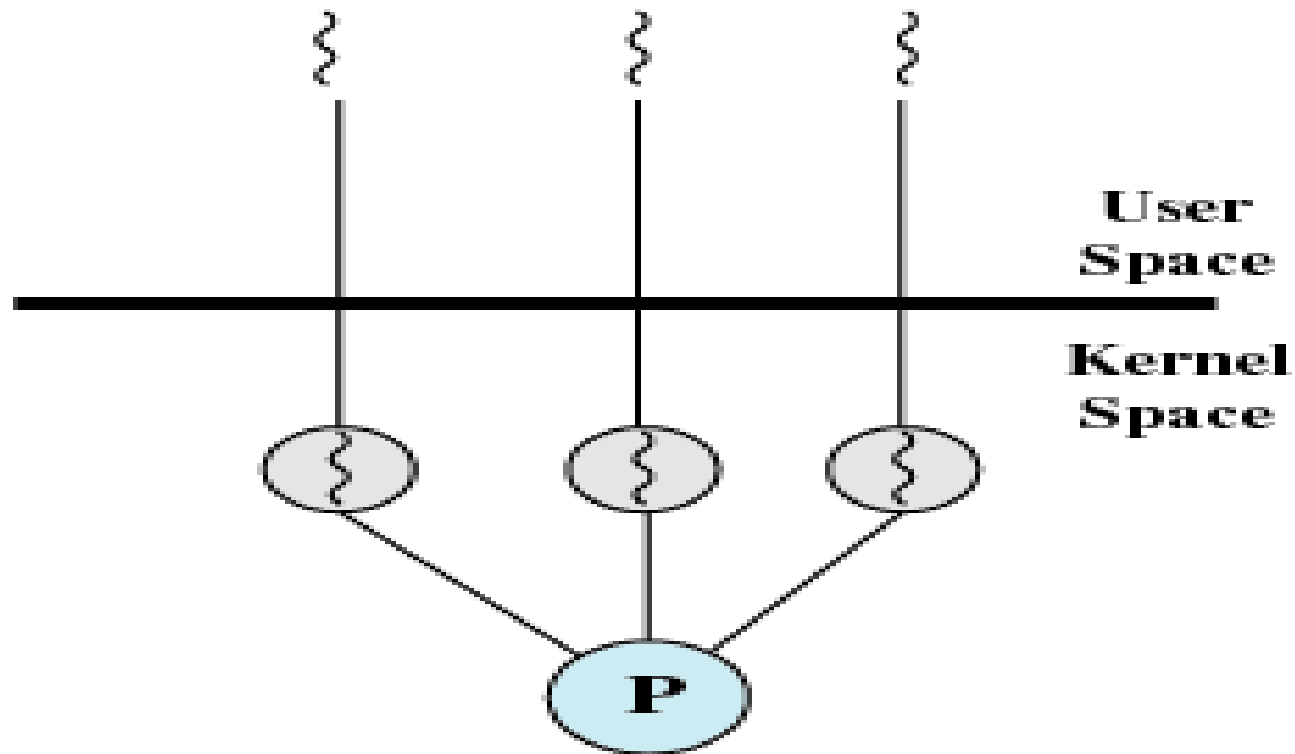


Kernel-Level Threads

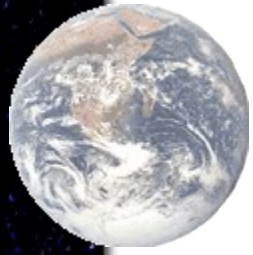
- No thread Library needed.
- Kernel aware of existence of thread
- Kernel maintains context information for the process and the threads
- Scheduling is done on a thread basis.
- Windows is an example of this approach



Kernel-Level Threads

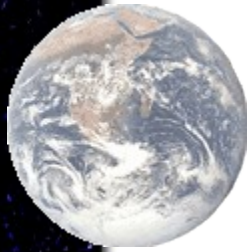


(b) Pure kernel-level



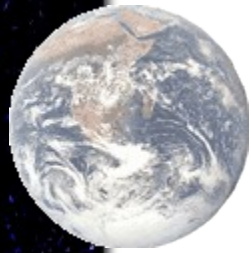
Advantages of KLT's

- Simultaneously schedule multiple threads from the same process on multiple processors
- If one thread is blocked, kernel can schedule another thread of the same process
- Kernel routines can be multithreaded



Disadvantages of KLT's

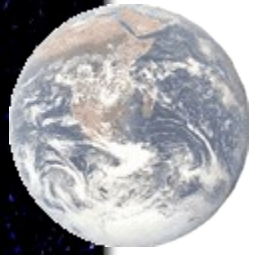
- Requires mode switch for thread transfer :
 - transfer of control from one thread to another within the same process requires a mode switch to the kernel.



VAX Running UNIX-Like Operating System

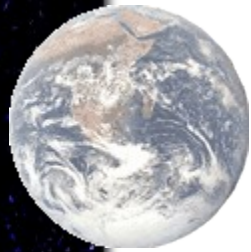
Table 4.1 Thread and Process Operation Latencies (μ s) [ANDE92]

Operation	User-Level Threads	Kernel-Level Threads	Processes
Null Fork	34	948	11,300
Signal Wait	37	441	1,840

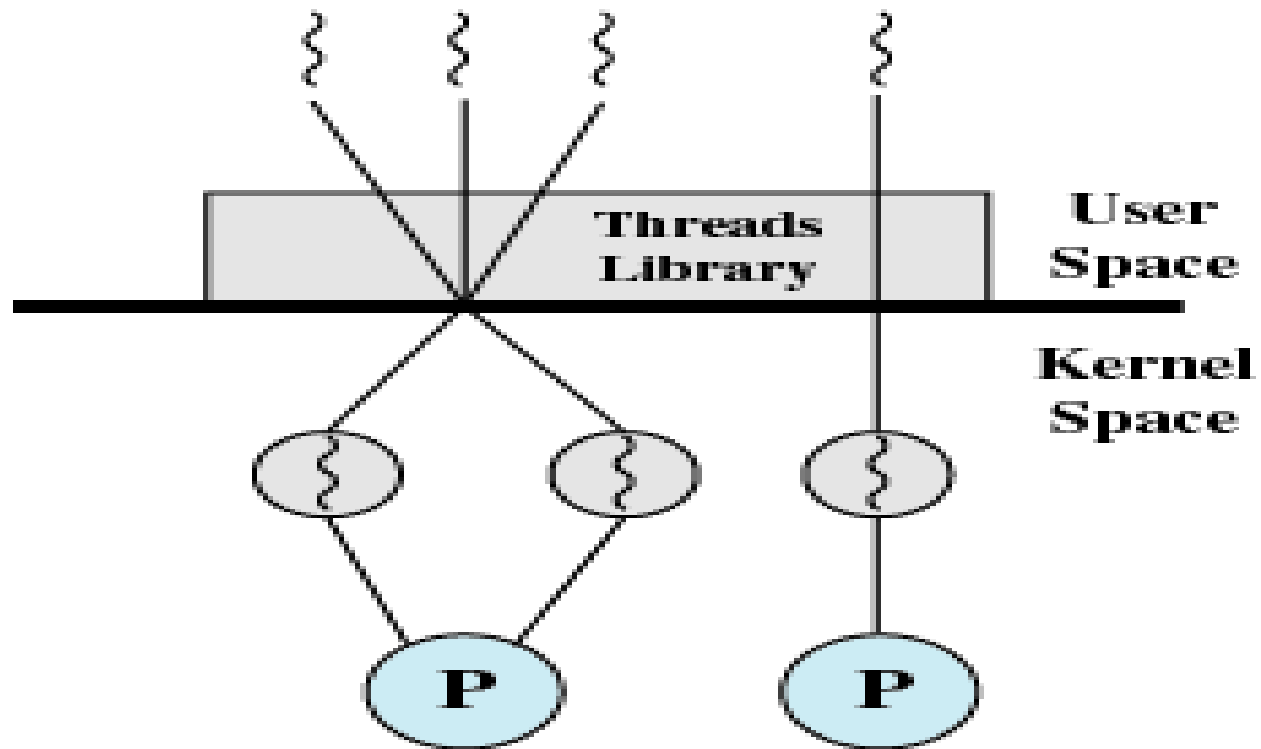


Combined Approaches

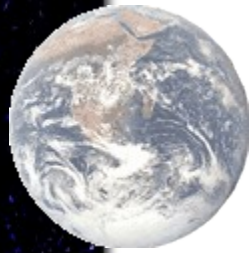
- Example is Solaris
- Thread creation done in the user space
- Bulk of scheduling and synchronization of threads within application



Combined Approaches



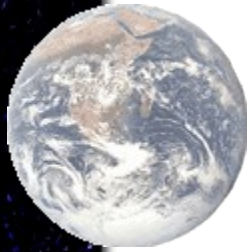
(c) Combined



Relationship Between Threads and Processes

Table 4.2 Relationship Between Threads and Processes

Threads:Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux OS/2, OS/390, MACH
1:M	A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems.	Ra (Clouds), Emerald
M:N	Combines attributes of M:1 and 1:M cases.	TRIX



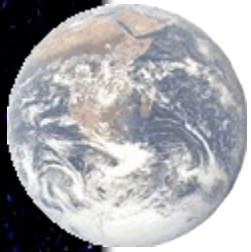
Categories of parallel processing architecture

- **Single Instruction Single Data (SISD) stream**
 - Single processor executes a single instruction stream to operate on data stored in a single memory
- **Single Instruction Multiple Data (SIMD) stream**
 - Each instruction is executed on a different set of data by the different processors
- **Multiple Instruction Single Data (MISD) stream**
 - A sequence of data is transmitted to a set of processors, each of which executes a different instruction sequence. Never implemented
- **Multiple Instruction Multiple Data (MIMD)**
 - A set of processors simultaneously execute different instruction sequences on different data sets

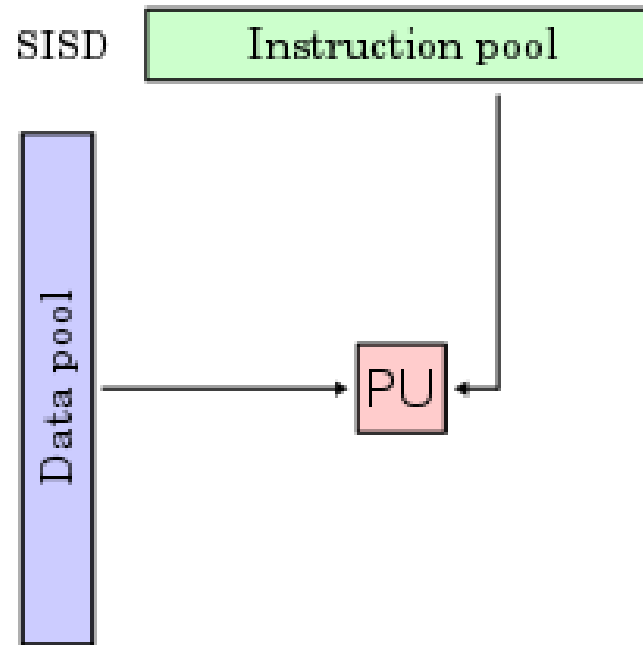


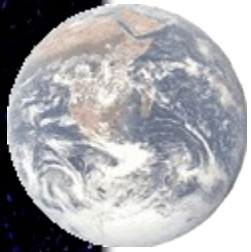
Categories of Computer Systems

- **SISD (Single Instruction, Single Data stream)**
 - Architecture in which a single processor (one CPU) executes exactly one instruction stream at a time
 - fetches or stores one item of data at a time to operate on data stored in a single memory unit.
 - Most of the CPU design, based on the von Neumann architecture, from the beginning till recent times are based on the SISD.
 - The SISD model is a typical non-pipelined architecture with the general-purpose registers, as well as dedicated special registers such as the Program Counter (PC), the Instruction Register (IR), Memory Address Registers (MAR) and Memory Data Registers (MDR).



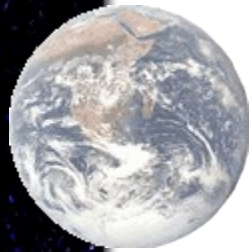
Categories of Computer Systems



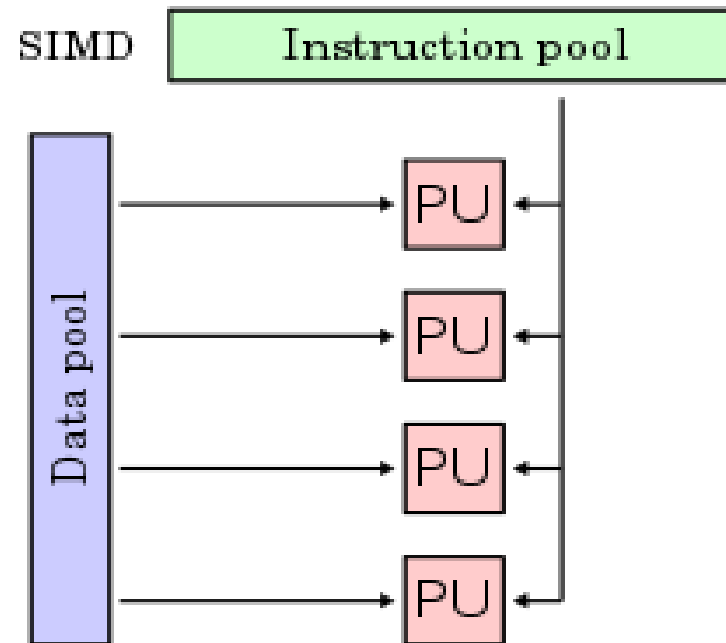


SIMD

- Architecture that have a
 - **single** control unit (CU)
 - **More than one** processing unit (PU)
 - Executes a single instruction stream over PUs, handled through the CU.
 - The CU generates the control signals for all of the PUs.
 - Executes the same operation on different data streams.
 - The SIMD architecture, in effect, is capable of achieving **data level parallelism** just like with **vector processor**.



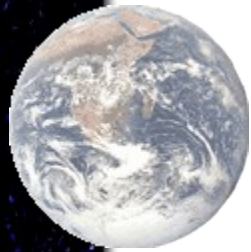
SIMD



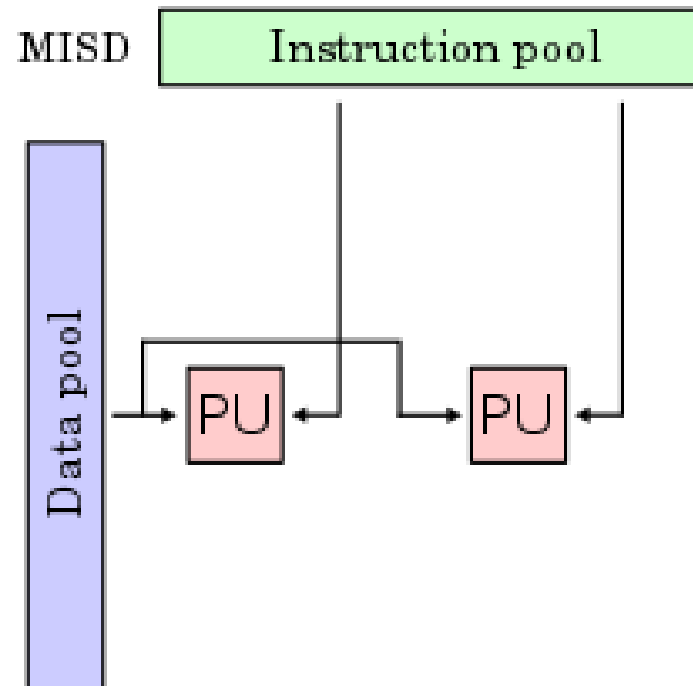


MISD

- Many functional units perform different operations
 - Executes different instructions on the same data set.
 - Mainly in the fault-tolerant computers.
 - Executing the same instructions redundantly
 - To detect and mask errors.



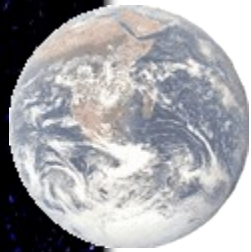
MISD



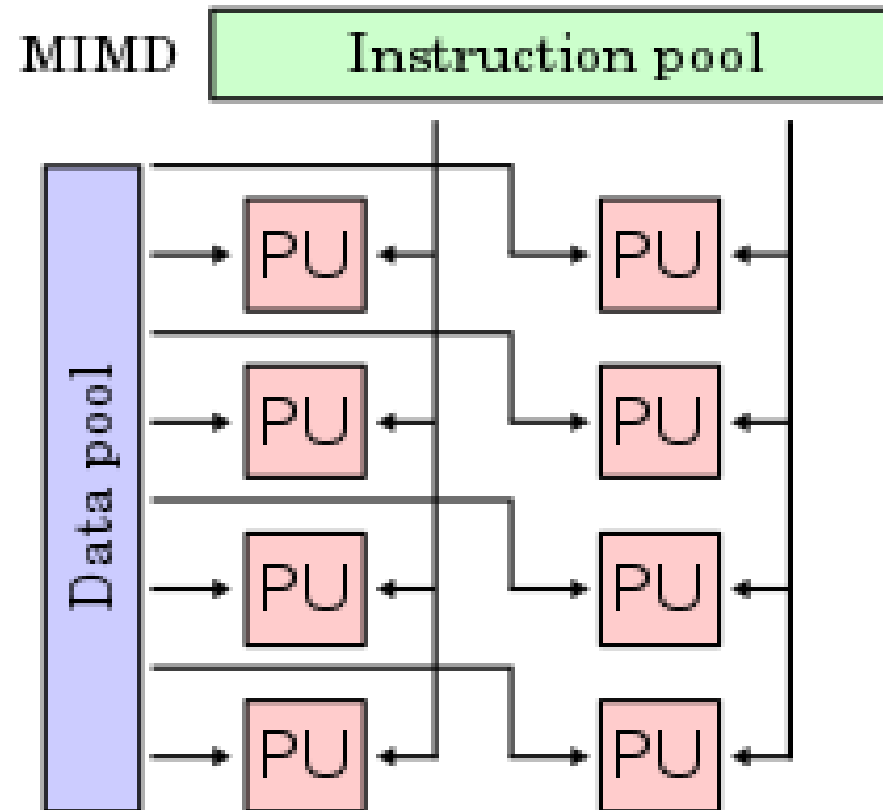


MIMD

- Typical of the computers with multiprocessors.
 - Each processor in a multiprocessor system can execute
 - asynchronously different set of the instructions independently on the different set of data units.
 - The MIMD based computer systems
 - can use the shared memory in a memory pool
 - or work using distributed memory across heterogeneous network computers in a distributed environment.
 - used in a number of application areas such as **computer-aided design/computer-aided manufacturing, simulation, modeling, communication switches etc.**



MISD



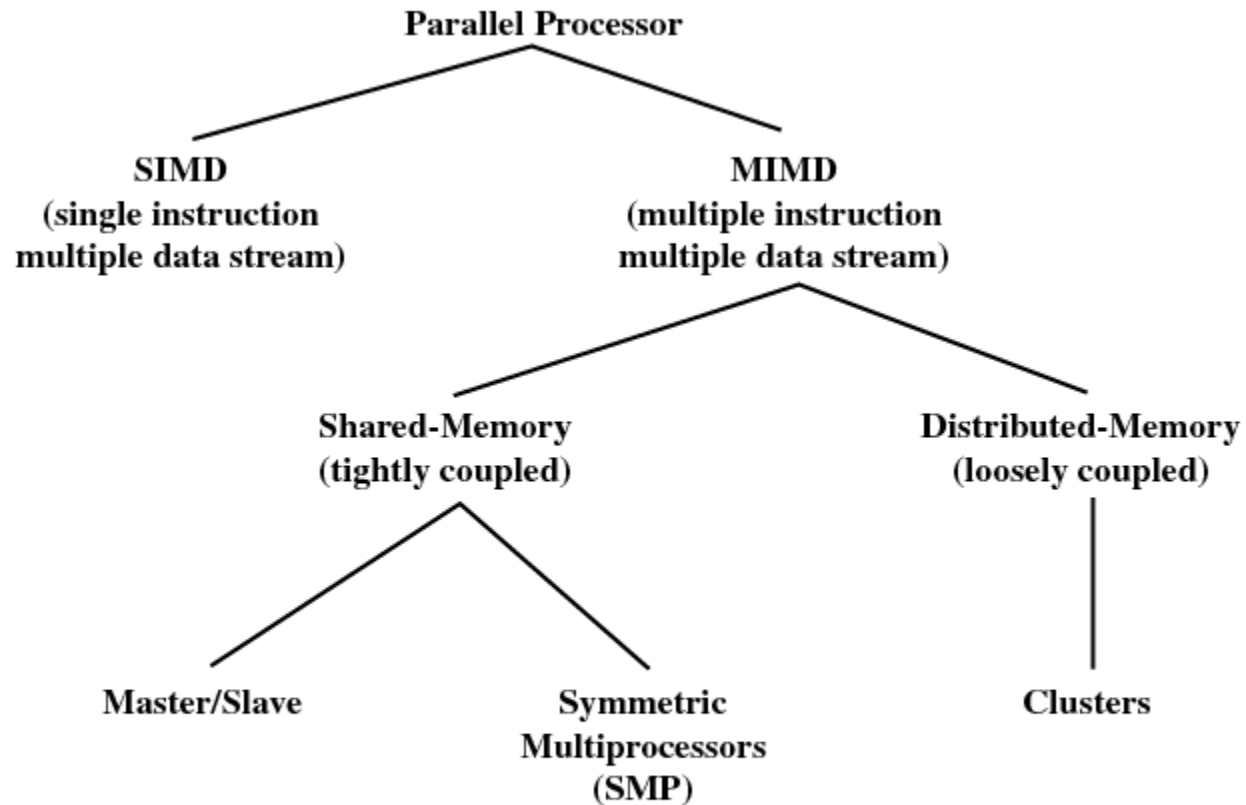
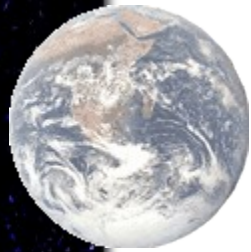


Figure 4.8 Parallel Processor Architectures



Symmetric Multiprocessing

- Kernel can execute on any processor
- Typically each processor does self-scheduling from the pool of available process or threads

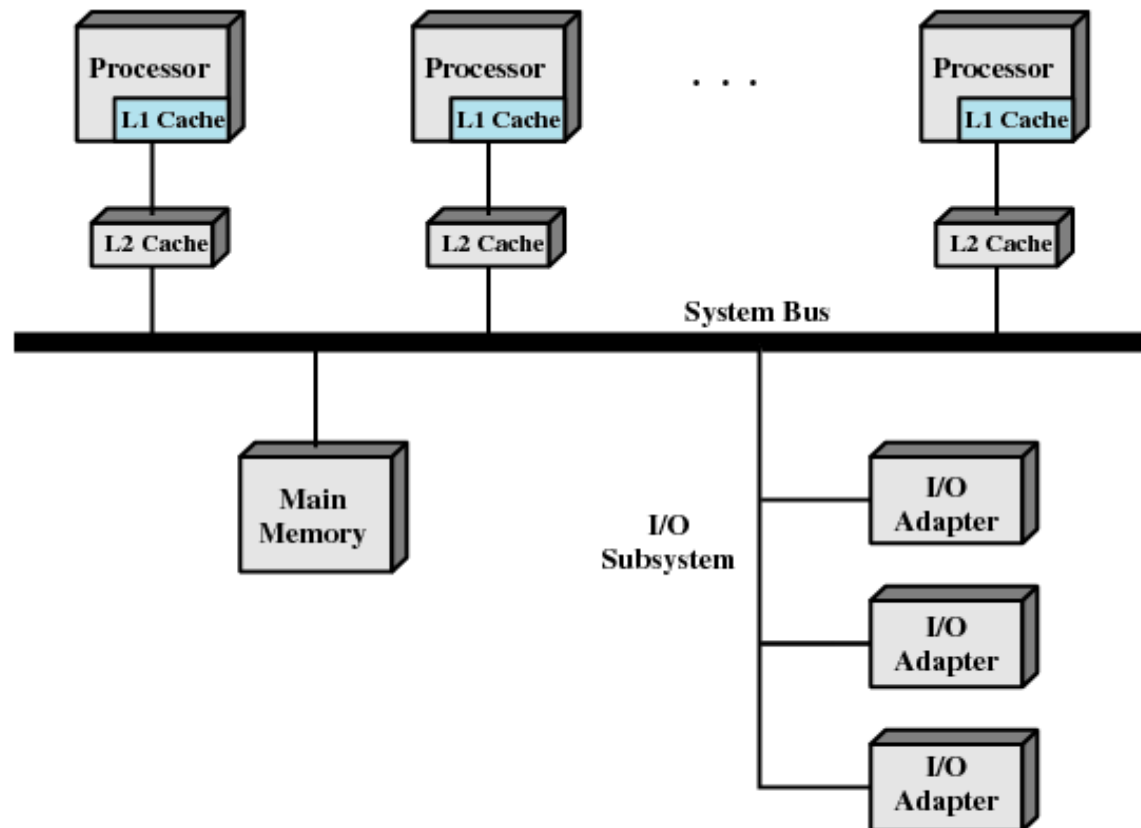
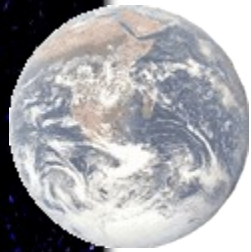
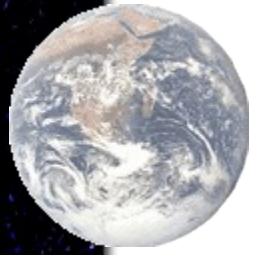
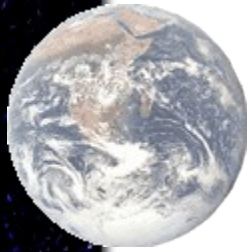


Figure 4.9 Symmetric Multiprocessor Organization



Multiprocessor Operating System Design Considerations

- Simultaneous concurrent processes or threads
- Scheduling
- Synchronization
- Memory management
- Reliability and fault tolerance



Microkernels

- Small operating system core
- Contains only essential core operating systems functions
- Many services traditionally included in the operating system are now external subsystems
 - Device drivers
 - File systems
 - Virtual memory manager
 - Windowing system
 - Security services

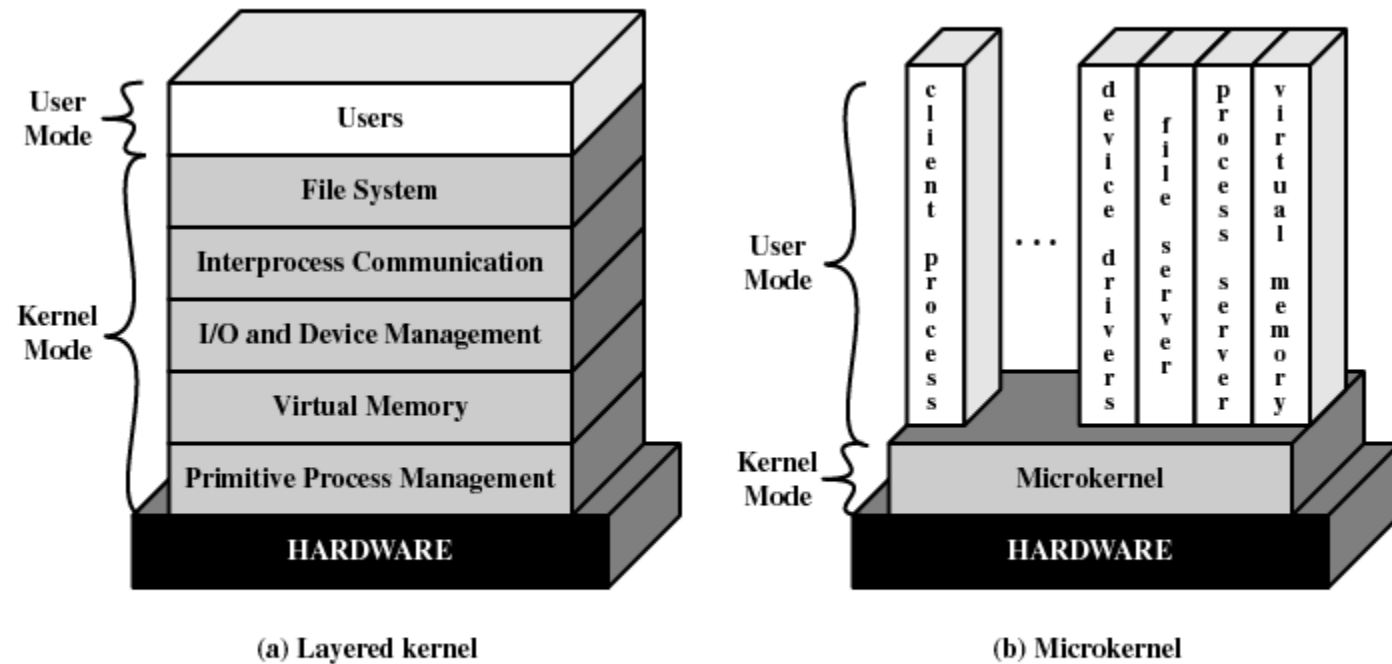
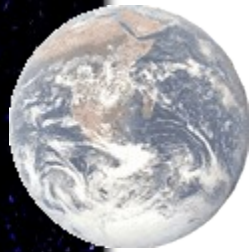


Figure 4.10 Kernel Architecture



Benefits of a Microkernel Organization

- Uniform interface on request made by a process
 - Don't distinguish between kernel-level and user-level services
 - All services are provided by means of message passing
- Extensibility
 - Allows the addition of new services
- Flexibility
 - New features added
 - Existing features can be subtracted



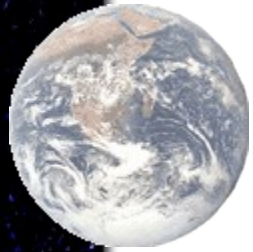
Benefits of a Microkernel Organization

- Portability
 - Changes needed to port the system to a new processor is changed in the microkernel - not in the other services
- Reliability
 - Modular design
 - Small microkernel can be rigorously tested



Benefits of Microkernel Organization

- Distributed system support
 - Message are sent without knowing what the target machine is
- Object-oriented operating system
 - Components are objects with clearly defined interfaces that can be interconnected to form software



Microkernel Design

- Low-level memory management
 - Mapping each virtual page to a physical page frame

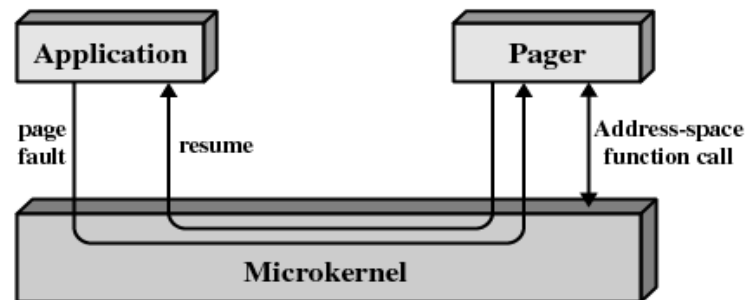
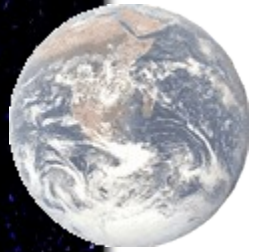
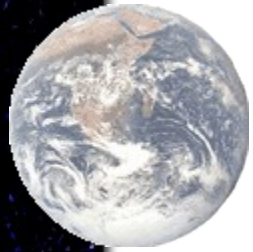


Figure 4.11 Page Fault Processing



Microkernel Design

- Interprocess communication
- I/O and interrupt management



Windows Processes

- Implemented as objects
- An executable process may contain one or more threads
- Both processes and thread objects have built-in synchronization capabilities

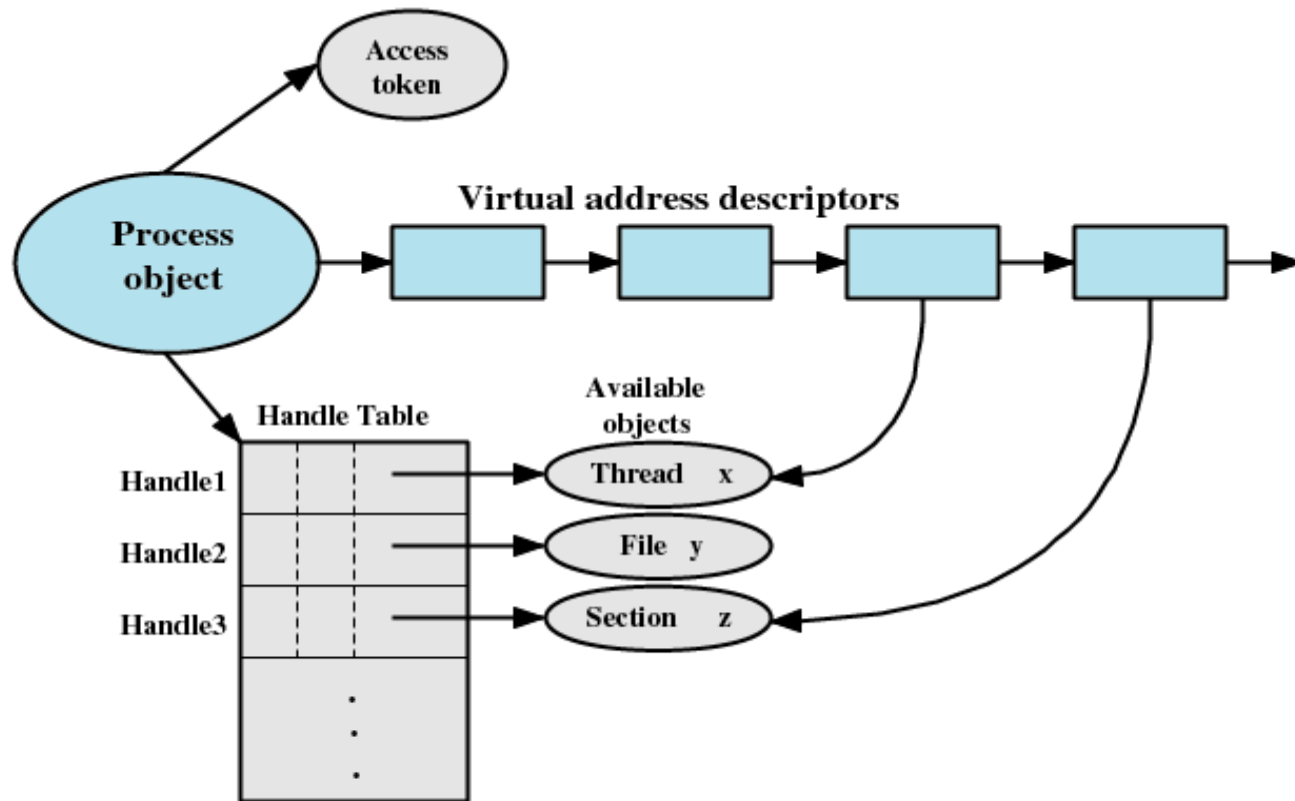
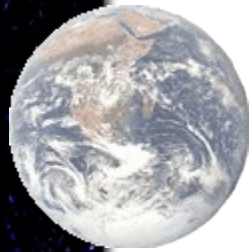
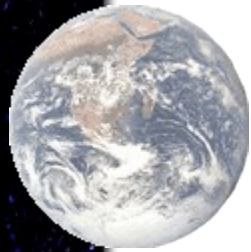
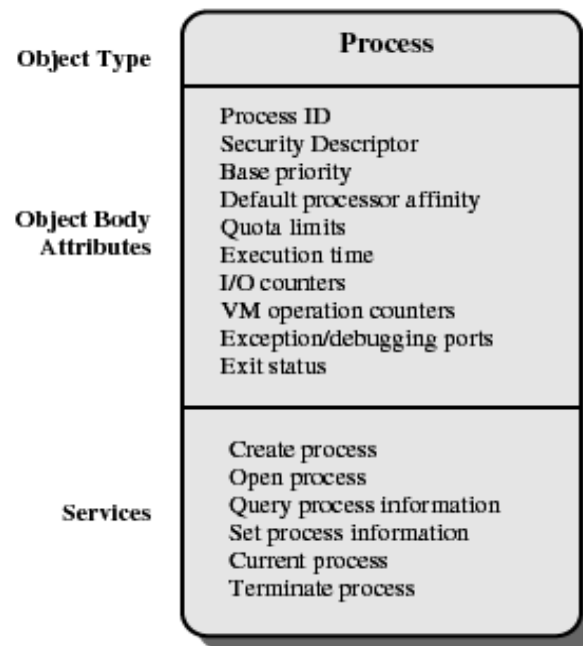


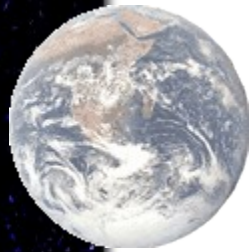
Figure 4.12 A Windows Process and Its Resources



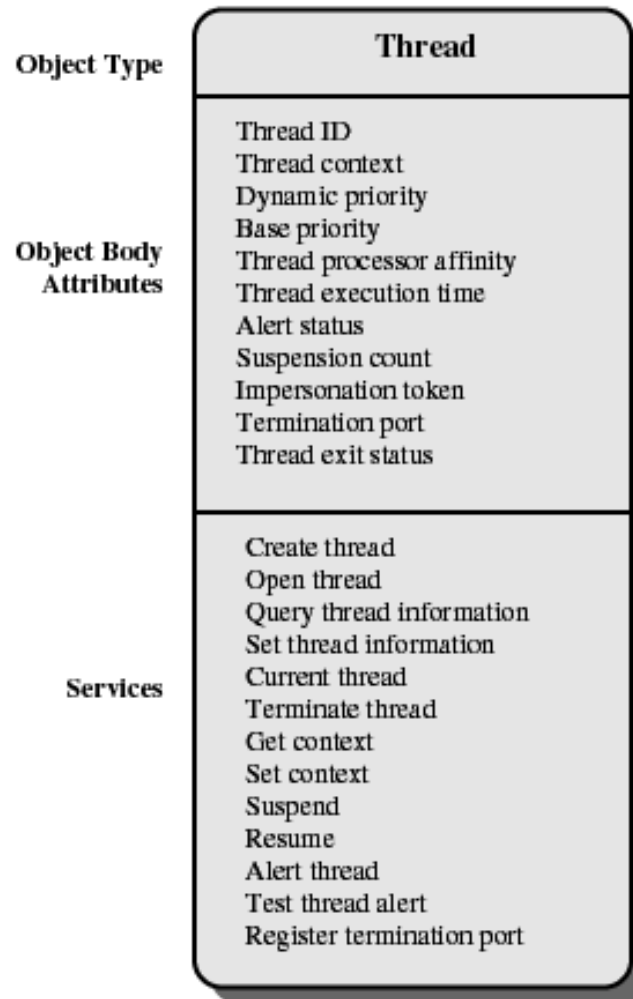
Windows Process Object



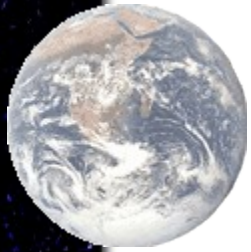
(a) Process object



Windows Thread Object



(b) Thread object



Windows 2000 Thread States

- Ready
- Standby
- Running
- Waiting
- Transition
- Terminated

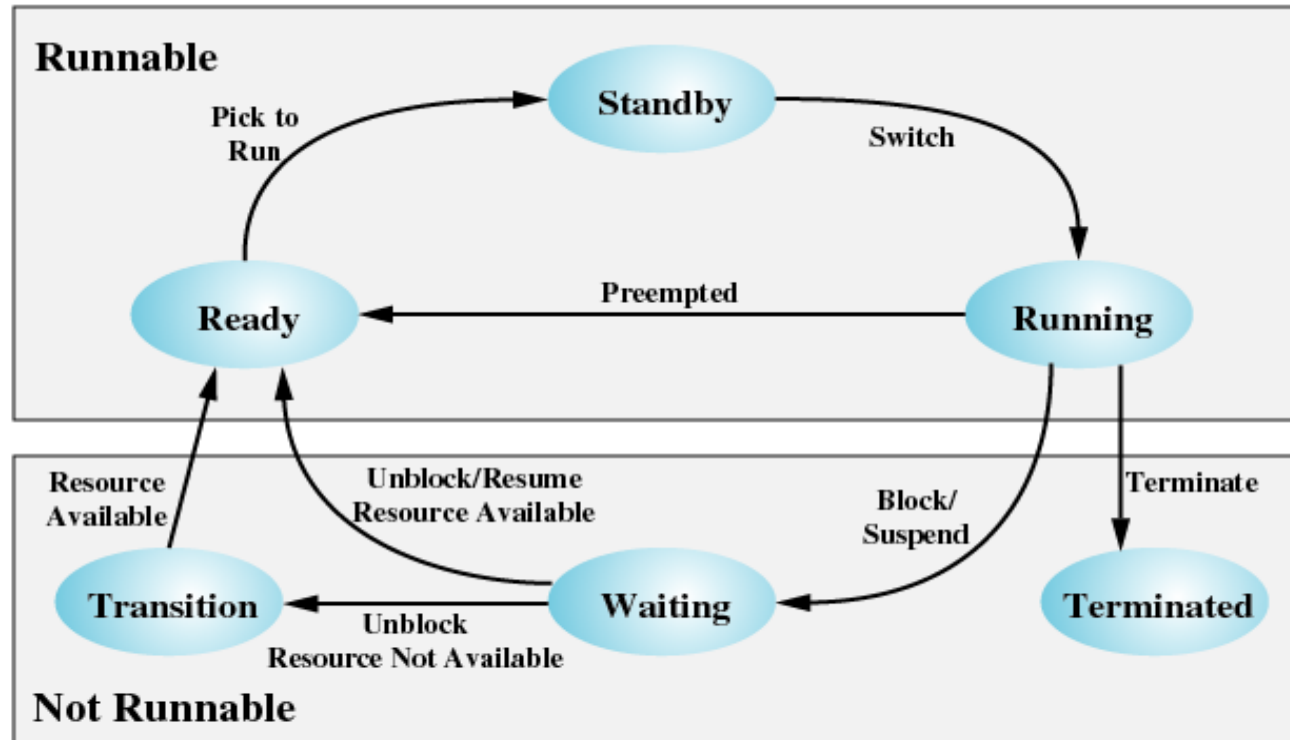
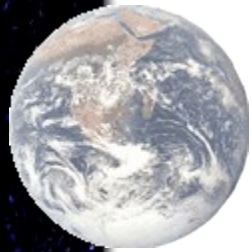


Figure 4.14 Windows Thread States



Solaris

- Process includes the user's address space, stack, and process control block
- User-level threads
- Lightweight processes (LWP)
- Kernel threads

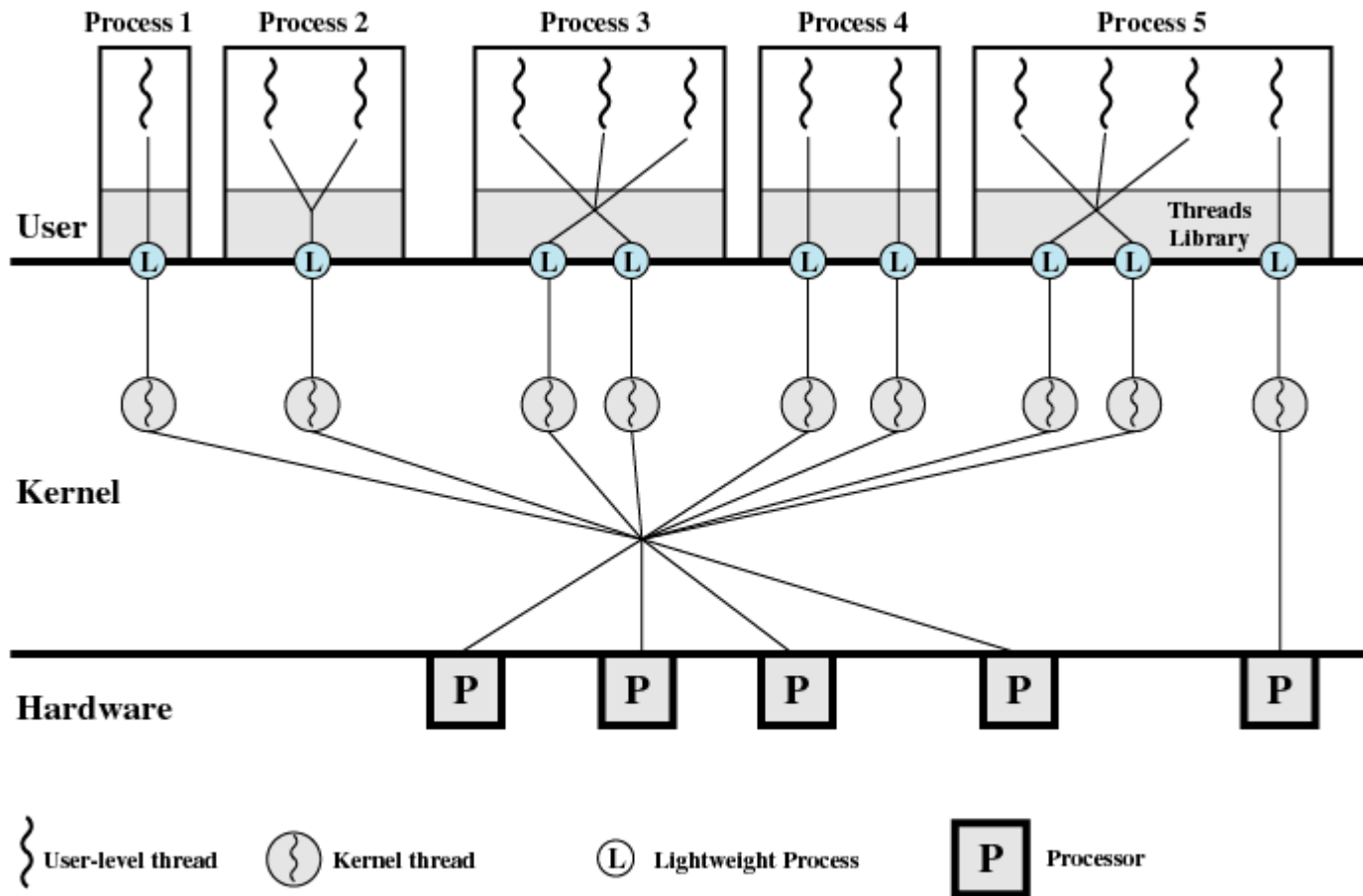
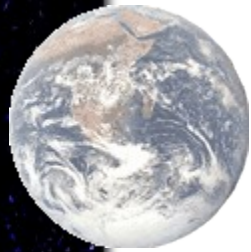
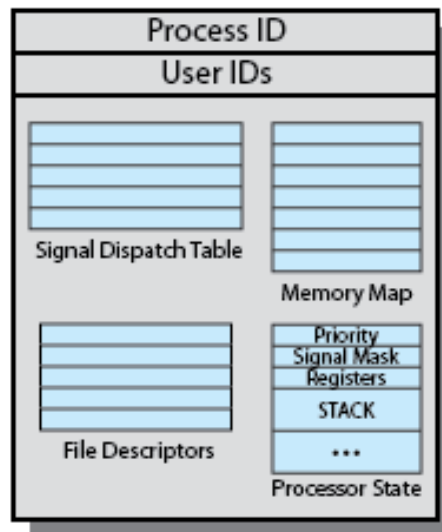


Figure 4.15 Solaris Multithreaded Architecture Example



UNIX Process Structure



Solaris Process Structure

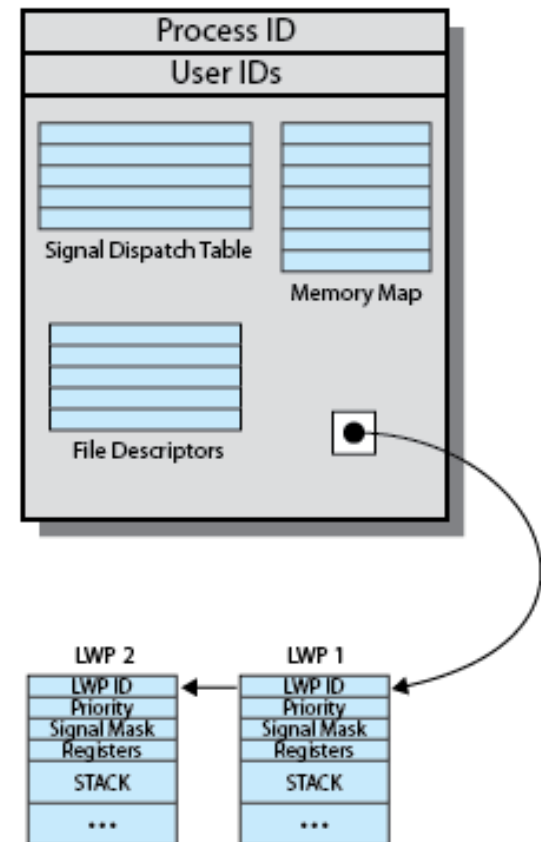


Figure 4.16 Process Structure in Traditional UNIX and Solaris [LEW196]



Solaris Lightweight Data Structure

- Identifier
- Priority
- Signal mask
- Saved values of user-level registers
- Kernel stack
- Resource usage and profiling data
- Pointer to the corresponding kernel thread
- Pointer to the process structure

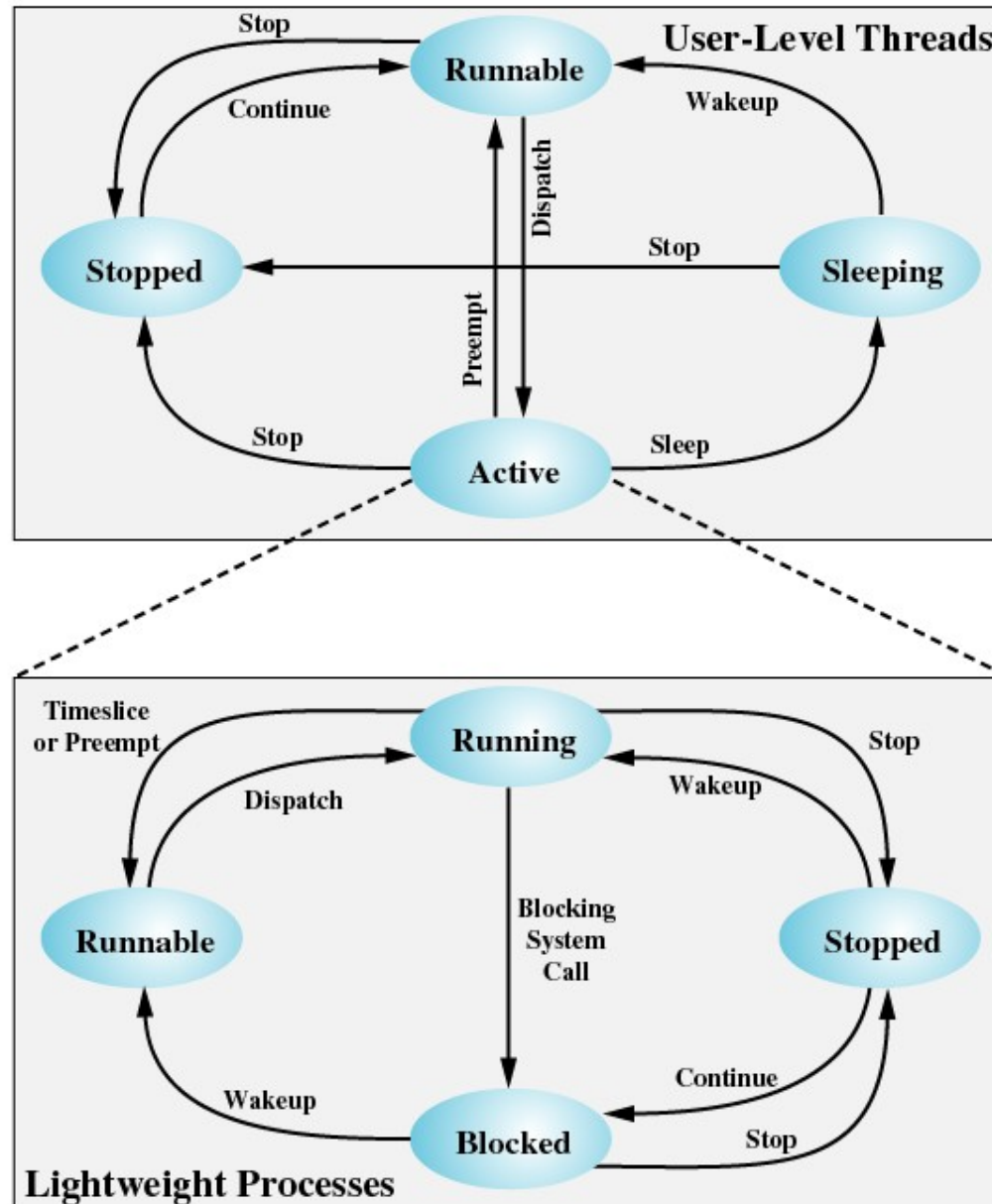
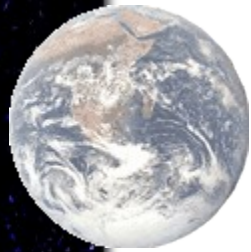
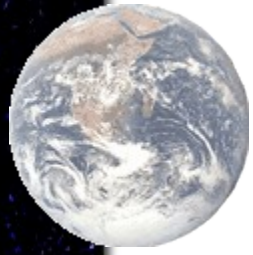


Figure 4.17 Solaris User-Level Thread and LWP States



Linux Task Data Structure

- State
- Scheduling information
- Identifiers
- Interprocess communication
- Links
- Times and timers
- File system
- Address space
- Processor-specific context



Linux States of a Process

- Running
- Interruptable
- Uninterruptable
- Stopped
- Zombie

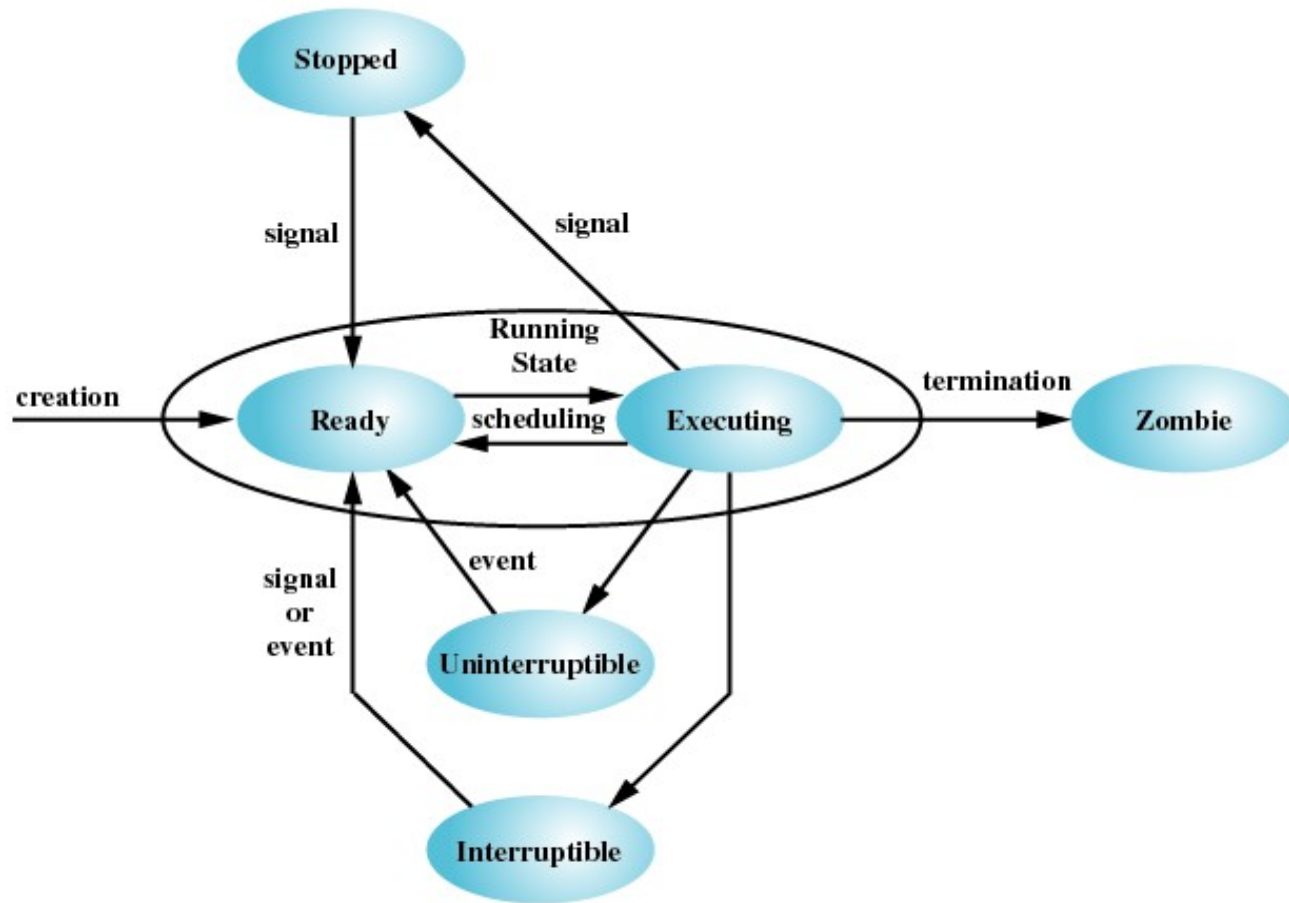
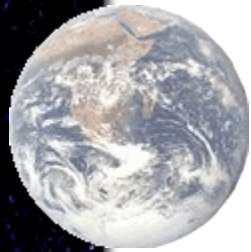


Figure 4.18 Linux Process/Thread Model